

Instituto Tecnológico Superior de Lerdo



Nombre del Alumno
Miguel Angel Luna Camacho

Numero de Control

252310055

Docente: Ing. Jesús Salas Marín

Actividad de Evaluacion C3 - Usuarios

Lerdo, Dgo., a 11 de Mayo de 2026

Documentación Técnica

Sistema de Usuarios con Herencia en Python

Descripción General del Proyecto

Este documento explica detalladamente la estructura y propósito de cada archivo del sistema de gestión de usuarios desarrollado en Python aplicando los principios de la Programación Orientada a Objetos: herencia, polimorfismo y encapsulamiento.

Archivo	Tipo de clase	Responsabilidad
usuario.py	Clase Base	Define los atributos y métodos comunes a todos los usuarios
admin.py	Clase Derivada	Modela un administrador con nivel de acceso y permisos totales
cliente.py	Clase Derivada	Modela un cliente con puntos de fidelidad y acceso a compras
invitado.py	Clase Derivada	Modela un usuario invitado con acceso mínimo al sistema
main.py	Programa Principal	Menú interactivo, lista de usuarios, polimorfismo y demostración

usuario.py - Clase Base

¿Para qué sirve?

Es la clase raíz del sistema. Define la estructura mínima que todo usuario debe tener: nombre y correo electrónico. También establece el contrato de comportamiento común: todos los usuarios pueden mostrarse, saludar y acceder al sistema. Las clases derivadas heredan estos elementos y los extienden sin repetir código.

Atributos

self.nombre: Almacena el nombre completo del usuario (tipo string)

self.email: Almacena el correo validado. Si el formato es incorrecto lanza un ValueError.

Métodos

__init__(nombre, email): Constructor. Asigna nombre y llama a `_validar_email()` antes de guardar el email.

_validar_email(email): Método privado. Usa expresión regular para verificar que el email tenga formato válido (usuario@dominio.ext). Lanza ValueError si no lo cumple.

mostrar_datos(): Imprime nombre y email del usuario en pantalla.

acceso_sistema(): Método base que indica acceso genérico. Es sobrescrito por cada subclase.

saludar(): Imprime un mensaje de bienvenida personalizado con el nombre del usuario.

Código completo

```
usuario.py > Usuario > saludar
1  import re
2
3  # clase base que representa a cualquier usuario del sistema
4  class Usuario:
5      def __init__(self, nombre, email):
6          self.nombre = nombre
7          self.email = self._validar_email(email) # desafio: validacion de email
8
9          # valida que el email tenga un formato correcto usando regex (regular expression)
10         def _validar_email(self, email):
11             patron = r'^[\w\.-]+@[\w\.-]+\.\w{2,}$'
12             if re.match(patron, email):
13                 return email
14             else:
15                 raise ValueError(f"Advertencia: Email invalido: '{email}'")
16
17         # muestra los datos basicos del usuario
18         def mostrar_datos(self):
19             print(f" Nombre : {self.nombre}")
20             print(f" Email  : {self.email}")
21
22         # metodo que cada subclase debe sobrescribir
23         def acceso_sistema(self):
24             print(f" [{self.nombre}] tiene acceso generico al sistema.")
25
26         # desafio: saludo personalizado
27         def saludar(self):
28             print(f" ¡Hola, {self.nombre}! Bienvenido al sistema.")
```

Conceptos aplicados

Encapsulamiento: `_validar_email()` es privado (prefijo `_`). No debe llamarse desde fuera.

Expresiones regulares: Se usa `re.match()` para validar que el email tenga un formato correcto.

Constructor: `__init__` Inicializa los atributos al crear cada objeto de tipo Usuario o subclase.

admin.py - Clase Derivada

¿Para qué sirve?

Representa a un administrador del sistema. Hereda todo de Usuario y agrega el atributo `nivel_acceso` (entero del 1 al 5) que determina sus privilegios. Sobrescribe `acceso_sistema()` y `mostrar_datos()` para reflejar que tiene acceso total y mostrar su nivel.

Atributos propios (además de los heredados)

self.nivel_acceso: Número entero (1–5) que indica el nivel de privilegios del administrador.

Métodos sobrescritos

__init__(nombre, email, nivel_acceso): Llama a `super().__init__()` para inicializar nombre y email, luego asigna `nivel_acceso`.

acceso_sistema(): Reemplaza el método base. Informa que el admin tiene acceso total e imprime su nivel.

mostrar_datos(): Llama a `super().mostrar_datos()` para imprimir nombre y email, luego agrega la línea del nivel de acceso.

Métodos sobrescritos

```
admin.py > Admin > mostrar_datos
1  from usuario import Usuario
2
3  # clase Admin: hereda de usuario y agrega nivel de acceso
4  class Admin(Usuario):
5      def __init__(self, nombre, email, nivel_acceso):
6          super().__init__(nombre, email)      # llama al constructor de usuario
7          self.nivel_acceso = nivel_acceso      # atributo propio de admin
8
9      # sobrescribe el metodo de la clase base
10     def acceso_sistema(self):
11         print(f" [{self.nombre}] ADMINISTRADOR – Nivel {self.nivel_acceso}: acceso total al sistema.")
12
13     # muestra datos incluyendo el atributo propio
14     def mostrar_datos(self):
15         super().mostrar_datos()                # reutiliza mostrar_datos() de usuario
16         print(f" Nivel de acceso: {self.nivel_acceso}")
```

¿Por qué se usa super()?

`super().__init__()` permite que Admin inicialice nombre y email usando el código ya escrito en Usuario (incluyendo la validación del email), sin repetirlo. De igual forma, `super().mostrar_datos()` reutiliza la impresión de nombre y email y Admin solo agrega la línea adicional del nivel de acceso.

cliente.py - Clase Derivada

¿Para qué sirve?

Modela a un cliente registrado de la plataforma. Hereda de Usuario y añade el atributo puntos que acumula el cliente a través de sus compras. Sobrescribe acceso_sistema() para redirigirlo a la zona de compras e informar sus puntos actuales.

Atributos propios (además de los heredados)

self.puntos: Número entero con los puntos de fidelidad acumulados. Valor por defecto: 0.

Métodos sobrescritos

__init__(nombre, email, puntos=0): Llama a super().__init__() y asigna los puntos. El valor predeterminado es 0.

acceso_sistema(): Indica que el cliente accede a la zona de compras y muestra sus puntos acumulados.

mostrar_datos(): Extiende mostrar_datos() del padre con super() y agrega los puntos de fidelidad.

Métodos sobrescritos

```
cliente.py > Cliente > mostrar_datos
1  from usuario import Usuario
2
3  # clase cliente: hereda de usuario y agrega puntos de fidelidad
4  class Cliente(Usuario):
5      def __init__(self, nombre, email, puntos=0):
6          super().__init__(nombre, email)      # llama al constructor de usuario
7          self.puntos = puntos                  # atributo propio de cliente
8
9      # sobrescribe el metodo de la clase base
10     def acceso_sistema(self):
11         print(f" [{self.nombre}] CLIENTE - Acceso a zona de compras. Puntos acumulados: {self.puntos}.")
12
13     # muestra datos incluyendo los puntos
14     def mostrar_datos(self):
15         super().mostrar_datos()                # reutiliza mostrar_datos() de usuario
16         print(f" Puntos de fidelidad: {self.puntos}")
```

Nota sobre el parámetro puntos=0

Al definir puntos=0 en el constructor, se establece un valor por defecto. Esto permite crear un cliente sin especificar puntos (Cliente("Ana","ana@mail.com")) y el atributo valdrá 0 automáticamente. Si se pasan puntos, ese valor se usa.

invitado.py - Clase Derivada

¿Para qué sirve?

Representa al usuario con menor nivel de acceso en el sistema: un visitante sin cuenta registrada. Hereda completamente de Usuario sin agregar atributos propios. Solo sobrescribe `acceso_sistema()` para restringir sus permisos al mínimo: únicamente puede ver contenido público.

Atributos propios

Invitado no agrega ningún atributo propio. Utiliza únicamente nombre y email heredados de Usuario. Esto refleja que un invitado tiene la información mínima necesaria.

Métodos sobrescritos

`__init__(nombre, email)`: Llama directamente a `super().__init__()`. Sin lógica adicional.

`acceso_sistema()`: Informa que el invitado solo puede ver contenido público. Acceso limitado.

Código completo

```
invitado.py > Invitado > acceso_sistema
1  from usuario import Usuario
2
3  # clase invitado: hereda de usuario con acceso limitado
4  class Invitado(Usuario):
5      def __init__(self, nombre, email):
6          super().__init__(nombre, email) #llama al constructor de usuario
7
8      # sobrescribe el metodo con permisos restringidos
9      def acceso_sistema(self):
10         print(f" [{self.nombre}] INVITADO – Solo puede ver contenido publico. Acceso limitado.")
```

¿Por qué sobrescribir si es la clase más simple?

Aunque Invitado no agrega atributos, sobrescribir `acceso_sistema()` es fundamental para el polimorfismo: al recorrer una lista mixta de usuarios y llamar `acceso_sistema()` en cada uno, Python ejecuta el método correcto según el tipo real del objeto. Sin sobrescritura, Invitado mostraría el mismo mensaje que la clase base.

main.py - Programa Principal

¿Para qué sirve?

Es el punto de entrada del sistema. Importa las cuatro clases, crea objetos de cada tipo, y presenta un menú interactivo al usuario. Contiene además la lista global de usuarios y demuestra el polimorfismo recorriendo esa lista con un bucle.

Estructura del archivo

Seccion / Funcion	Descripcion
Importaciones	Trae las clases Admin, Cliente e Invitado (que a su vez importan Usuario).
Lista usuarios[]	Lista global que almacena los objetos creados. Inicia con 1 Admin, 1 Cliente y 1 Invitado.
mostrar_todos()	Recorre la lista e imprime tipo, datos y nivel de acceso de cada usuario.
demo_polimorfismo()	Llama a acceso_sistema() en todos los usuarios: mismo método, distinta respuesta
agregar_usuario()	Menú para crear un nuevo usuario de cualquier tipo y agregarlo a la lista.
saludar_todos()	Llama a saludar() en cada usuario de la lista.
menu()	Bucle while con el menú principal. Coordina todas las funciones anteriores.
if __name__==...	Punto de entrada. Llama a menu() solo cuando el archivo se ejecuta directamente.

Fragmento clave – Polimorfismo

```
for u in usuarios: # misma llamada, comportamiento distinto segun el tipo real
    u.acceso_sistema() # admin, cliente o invitado responden diferente
```

Diagrama de flujo del menú

1. **Llama a mostrar_todos():** imprime tipo, datos y acceso de cada usuario.
2. **Llama a agregar_usuario():** pide tipo, nombre, email y datos extra; valida y agrega a la lista.
3. **Llama a demo_polimorfismo():** recorre la lista y muestra el mensaje de acceso de cada tipo.
4. **Llama a saludar_todos():** imprime el saludo personalizado de cada usuario.
5. Sale del programa con un mensaje de despedida.

Pruebas

Opción 1 – Mostrar todos los usuarios

Se selecciona la opción 1 del menú principal. El sistema recorre la lista de usuarios e imprime el tipo de clase de cada objeto (Admin, Cliente, Invitado), sus datos personales mediante `mostrar_datos()` y su nivel de acceso mediante `acceso_sistema()`. Se puede observar cómo cada tipo responde de forma distinta al mismo llamado de método, lo que demuestra el principio de polimorfismo.

```
PS C:\xampp\htdocs\Actividad de Evaluacion C3 - Usuarios> & C:/Users/migue/AppData/Local/Programs/Python/Python313/python.exe "c:/xampp\htdocs/Actividad de Evaluacion C3 - Usuarios/main.py"

SISTEMA DE GESTION DE USUARIOS

1. Mostrar todos los usuarios
2. Agregar nuevo usuario
3. Demostracion polimorfismo
4. Saludar a todos
5. Salir

Elige una opcion: 1

=====
LISTA DE USUARIOS DEL SISTEMA
=====

▶ Tipo: Admin
Nombre : Lhya Hernandez
Email  : lhya@admin.com
Nivel de acceso: 5
[Lhya Hernandez] ADMINISTRADOR – Nivel 5: acceso total al sistema.

▶ Tipo: Cliente
Nombre : María Rivera
Email  : maria@cliente.com
Puntos de fidelidad: 320
[María Rivera] CLIENTE – Acceso a zona de compras. Puntos acumulados: 320.

▶ Tipo: Invitado
Nombre : Miguel Sin Cuenta
Email  : miguel@temp.com
[Miguel Sin Cuenta] INVITADO – Solo puede ver contenido publico. Acceso limitado.
```

Opción 2 – Agregar nuevo usuario

Se selecciona la opción 2. El sistema solicita el tipo de usuario a crear (Admin, Cliente o Invitado), luego pide nombre y email. Si el email no cumple el formato válido, la validación con expresión regular lanza un `ValueError` y el usuario no se agrega. Si los datos son correctos, el nuevo objeto se instancia con su clase correspondiente y se añade a la lista global `usuarios[]`. Esta prueba evidencia el uso del constructor `__init__` y la validación de datos.

```
SISTEMA DE GESTION DE USUARIOS

1. Mostrar todos los usuarios
2. Agregar nuevo usuario
3. Demostracion polimorfismo
4. Saludar a todos
5. Salir

Elige una opcion: 2

— AGREGAR USUARIO —
1) Admin
2) Cliente
3) Invitado
Tipo: 3
Nombre: Pucca
Email: pucca@lol.com
✓ Usuario 'Pucca' agregado correctamente.
```

Opción 3 – Demostración de polimorfismo

Se selecciona la opción 3. El programa recorre la lista de usuarios con un bucle for y llama a `acceso_sistema()` en cada objeto. Aunque la llamada es idéntica para todos, cada clase ejecuta su propia versión del método gracias a la sobreescritura: el Admin muestra su nivel de privilegio, el Cliente sus puntos y la zona de compras, y el Invitado indica su acceso restringido. Esto demuestra el polimorfismo en acción.

```
SISTEMA DE GESTION DE USUARIOS

1. Mostrar todos los usuarios
2. Agregar nuevo usuario
3. Demostracion polimorfismo
4. Saludar a todos
5. Salir

Elige una opcion: 3

=====
DEMOSTRACIÓN DE POLIMORFISMO
=====
Llamando a acceso_sistema() en cada objeto:
[Lhya Hernandez] ADMINISTRADOR – Nivel 5: acceso total al sistema.
[María Rivera] CLIENTE – Acceso a zona de compras. Puntos acumulados: 320.
[Miguel Sin Cuenta] INVITADO – Solo puede ver contenido publico. Acceso limitado.
[Pucca] INVITADO – Solo puede ver contenido publico. Acceso limitado.
```

Opción 4 – Saludar a todos

Se selecciona la opción 4. El sistema llama al método `saludar()` en cada usuario de la lista. Este método es heredado directamente de la clase base `Usuario` sin ser sobrescrito por ninguna subclase, lo que demuestra la reutilización de código mediante herencia: un único método definido en el padre funciona correctamente para Admin, Cliente e Invitado.

```
SISTEMA DE GESTION DE USUARIOS

1. Mostrar todos los usuarios
2. Agregar nuevo usuario
3. Demostracion polimorfismo
4. Saludar a todos
5. Salir

Elige una opcion: 4

=====
SALUDOS
=====
¡Hola, Lhya Hernandez! Bienvenido al sistema.
¡Hola, María Rivera! Bienvenido al sistema.
¡Hola, Miguel Sin Cuenta! Bienvenido al sistema.
¡Hola, Pucca! Bienvenido al sistema.
```

Opción 5 – Salir

Se selecciona la opción 5. El bucle while del menú termina su ejecución al cumplirse la condición de salida, imprimiendo un mensaje de despedida. El programa finaliza correctamente sin errores, lo que confirma que el flujo de control del menú principal funciona según lo diseñado.

```
SISTEMA DE GESTION DE USUARIOS

1. Mostrar todos los usuarios
2. Agregar nuevo usuario
3. Demostracion polimorfismo
4. Saludar a todos
5. Salir

Elige una opcion: 5

¡Hasta luego!
```

Respuestas a las Preguntas de la Actividad

a) ¿Qué ventajas ofrece la herencia frente a repetir código?

La herencia permite definir atributos y métodos comunes (nombre, email, mostrar_datos, saludar) una sola vez en la clase base Usuario, y todas las subclases los reciben automáticamente. Si se requiere modificar la validación del email, basta con cambiar `_validar_email()` en `usuario.py` y las tres subclases se actualizan sin tocarlas. Reduce errores, facilita el mantenimiento y hace el código más legible y escalable.

b) ¿Qué métodos fueron heredados?

Los métodos `mostrar_datos()`, `acceso_sistema()`, `saludar()` y `_validar_email()` son definidos en Usuario y heredados por Admin, Cliente e Invitado. Admin y Cliente además los extienden usando `super()` para no perder la lógica del padre; Invitado hereda `saludar()` sin cambios.

c) ¿Por qué sobrescribir `acceso_sistema()`?

Porque cada tipo de usuario tiene permisos distintos: un Admin accede a todo el sistema con su nivel de privilegio, un Cliente solo a su zona de compras con sus puntos, y un Invitado únicamente al contenido público. Sobrescribir el método permite que cada clase exprese su propio comportamiento sin cambiar la interfaz externa, lo que es la base del polimorfismo.

d) ¿Diferencia entre atributo heredado y atributo propio?

Los atributos `nombre` y `email` son heredados: se definen en Usuario y todos los hijos los poseen automáticamente sin declaración adicional. Los atributos propios (`nivel_acceso` en Admin, `puntos` en Cliente) se declaran dentro del constructor de la subclase y solo pertenecen a esa clase; otras clases no los conocen ni los utilizan.

e) ¿Qué pasaría si las clases estuvieran separadas sin herencia?

Habría que duplicar los atributos `nombre` y `email`, la validación de email y los métodos `mostrar_datos()` y `saludar()` en cada uno de los tres archivos. Cualquier corrección requeriría cambios en múltiples archivos con riesgo de inconsistencias. Además se perdería el polimorfismo: no sería posible recorrer una lista mixta de usuarios con el mismo método `acceso_sistema()`, ya que cada clase sería completamente independiente.